Operating Systems

Session 6: semaphores and the other synchronization tools

process management: the lab exercises, worked step by step

Marius Dumitrașcu

UTM, FCIM / FAF · marius@cybercor.org · Telegram +40753210000 · Academic year 2026-2027

A course in two pillars: study the great operating systems, and build a small one.

TODAY'S QUESTION

Two threads, one variable: who wins?

- **Today the lecture is the lab.** We work, one by one, the exercises of Labs 11, 12 and 13, with their real output.
- **Question 1:** why do two threads that add to the same counter lose additions?
- **Question 2:** which tools does an operating system give us to stop it? (mutex, semaphore, condition variable)
- **Question 3:** how does a kernel build a semaphore? (you build one in xv6 today)

THE PLAN OF THE DAY

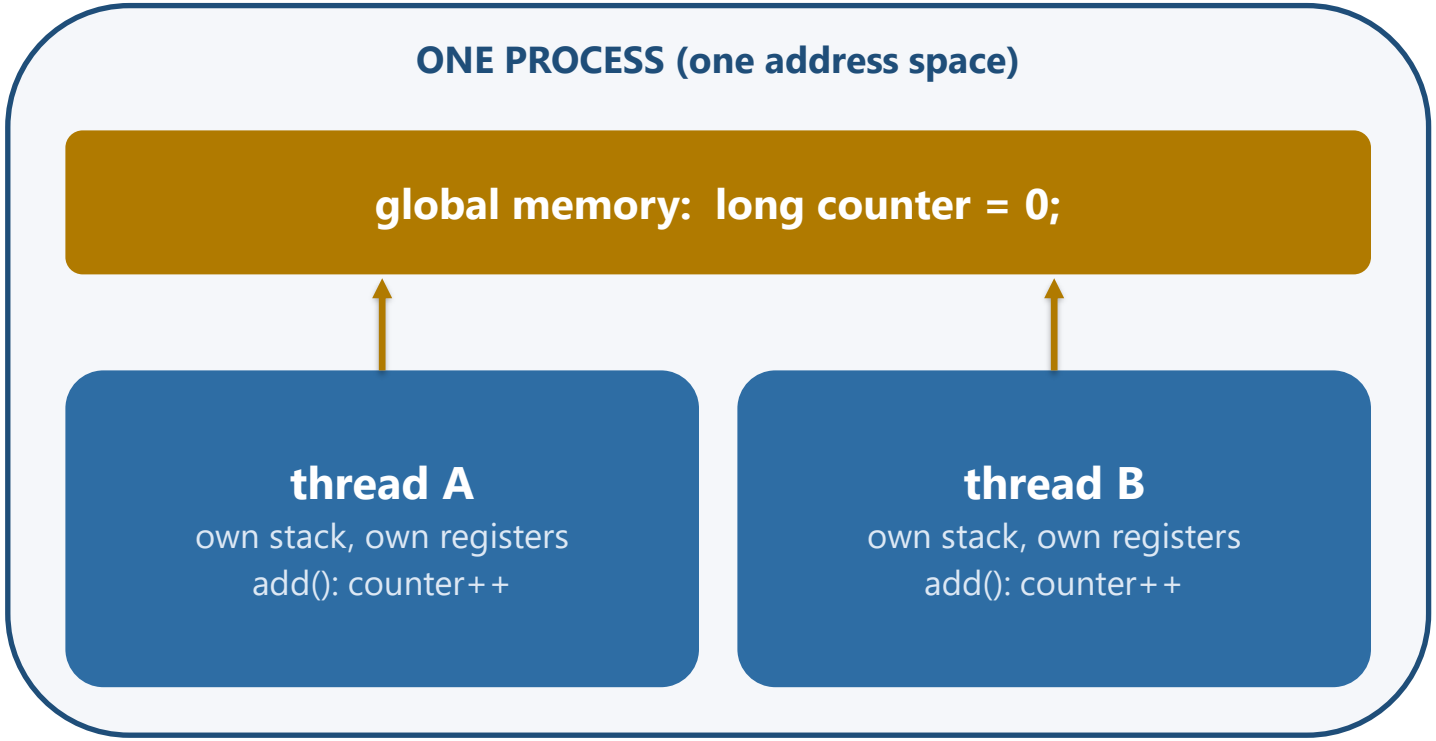
Eight exercises, the same as in the lab

1	`race.c`	the problem
2	`mutex.c`	the lock
3	`sem.c`	semaphore = lock
4	`order.c`	semaphore = signal
5	`parking.c`	semaphore = counter
6	`prodcons.c`	all three together
7	`condvar.c` `deadlock.c`	more tools, one trap
8	`kernel/sem.c`	build it in xv6

Lab 11 = exercises 1 to 5. Lab 12 = exercises 6, 7 (Linux). Lab 13 = exercise 8 (xv6).

BEFORE WE START

One process, two threads, one shared variable



pthread_create(&a, NULL, add, NULL);
starts a thread running add()

pthread_join(a, NULL);
waits for it to finish

Threads share memory. That is their power and their danger.

WHAT IT MEANS, CONCRETELY

What is a thread?

A process is a running program with its own memory. fork makes a second one, with a COPY of the memory.

A thread is a second flow of execution INSIDE the same process. It has its own stack and registers, but the global variables, the heap and the open files are the same for all threads.

Why threads? To use several CPU cores on one job, without copying the data. A browser, a database, a game: all threads.

TERMS

pthread_create: start a thread running a function

pthread_join: wait for a thread to finish

-pthread: the gcc flag that links the thread library

Remember: **Processes have separate memory; threads share it.**

QUESTION 1

Why do two threads lose work?

Exercise 1: race.c, two threads, one counter, one million additions each.

The code · the prediction · the real output · where the additions go · the words

EXERCISE 1 · STEP 1

race.c: two threads, one counter

C (race.c)

```c
long counter = 0;

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        counter++;
    return NULL;
}

pthread_create(&a, NULL, add, NULL);
pthread_create(&b, NULL, add, NULL);
pthread_join(a, NULL);
pthread_join(b, NULL);
printf("counter = %ld\n", counter);
```

- **N:** 1 000 000, so the right answer is 2 000 000
- **add():** both threads run the same function
- **counter:** a global: ONE copy, shared
- **gcc:** gcc -Wall -pthread -o race race.c

QUESTION

What does ./race print?

1	2	3	4
counter = 2000000	less than 2000000	more than 2000000	it crashes

THE ANSWER

What does ./race print?

1	2	3	4
counter = 2000000	**less than 2000000**	more than 2000000	it crashes

Less, and a different number at every run: some additions are lost.

EXERCISE 1 · STEP 2

The real output, three runs

terminal (Linux, real)

```
$ ./race
counter = 1050574 (expected 2000000)
$ ./race
counter = 1096707 (expected 2000000)
$ ./race
counter = 1146730 (expected 2000000)
```

- **expected:** 2 000 000
- **got:** about 1 050 000 to 1 150 000
- **lost:** almost half of the additions
- **every run:** a different number

QUESTION 1 · STEP 3

Where the lost additions go

time	thread A	thread B	counter	registers
1	load r = counter		5	A: r = 5
2		load r = counter	5	B: r = 5
3	add r = r + 1		5	A: r = 6
4		add r = r + 1	5	B: r = 6
5	store counter = r		6	
6		store counter = r	6	

Two threads added 1, the counter grew by 1: one addition is lost.

WHAT IT MEANS, CONCRETELY

Race condition and critical section

A race condition means the result depends on the order in which threads happen to run. The code is correct for one thread and wrong for two.

The critical section is the piece of code that touches the shared data: here, counter++. Only one thread may be in it at a time.

Why it is hard The bug does not show every time. It depends on timing, cores, load: a test can pass a thousand times.

TERMS

race condition: result depends on timing

critical section: code touching shared data

atomic: happens as one indivisible step

Remember: **counter++ is three steps, not one: two threads can interleave them and lose work.**

TRUE OR FALSE?

The race

1. counter++ is a single CPU instruction.

2. Running race.c many times always gives the same wrong number.

3. With only one thread there is no race.

THE ANSWERS

The race

1 **counter++ is a single CPU instruction.**

FALSE. Load, add, store: three steps that another thread can cut in between.

2 **Running race.c many times always gives the same wrong number.**

FALSE. The interleaving changes at every run, so the number changes too.

3 **With only one thread there is no race.**

TRUE. Nobody can cut in: the three steps always finish together.

QUESTION

Why do two threads lose additions?

The answer uses three words:

`load` `store` `critical section`

THE ANSWER

Why do two threads lose additions?

1. counter++ is three steps: load the value, add 1, store it back.
2. Two threads can both load the same value; both store the same result: one addition is lost.
3. The cure: make counter++ a critical section, one thread at a time.

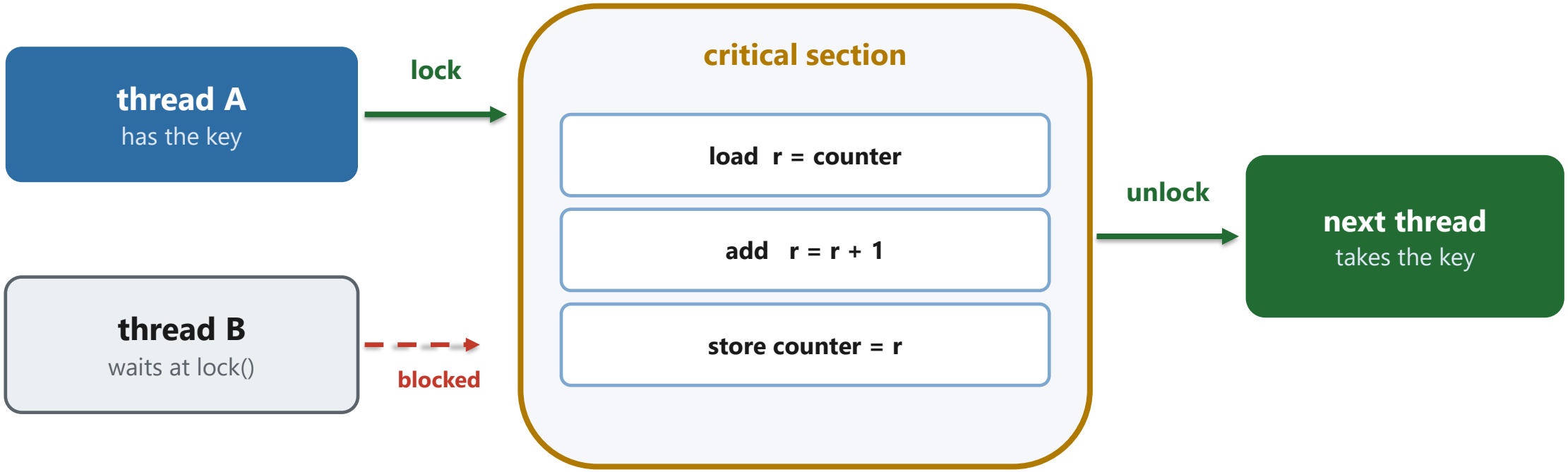
QUESTION 2

Which tools stop it?

mutex · semaphore as a lock · as a signal · as a counter · producer and consumer · condition variable · deadlock

QUESTION 2 · STEP 1

The fix: a door with one key



Mutual exclusion: at most one thread inside the critical section.

EXERCISE 2

mutex.c: lock around counter++

C (mutex.c)

```c
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

terminal (Linux, real)

```
$ ./mutex
counter = 2000000 (expected 2000000)
$ ./mutex
counter = 2000000 (expected 2000000)
```

Two lines added: the answer is exact, every time.

EXERCISE 2 · THE PRICE

Correct is slower

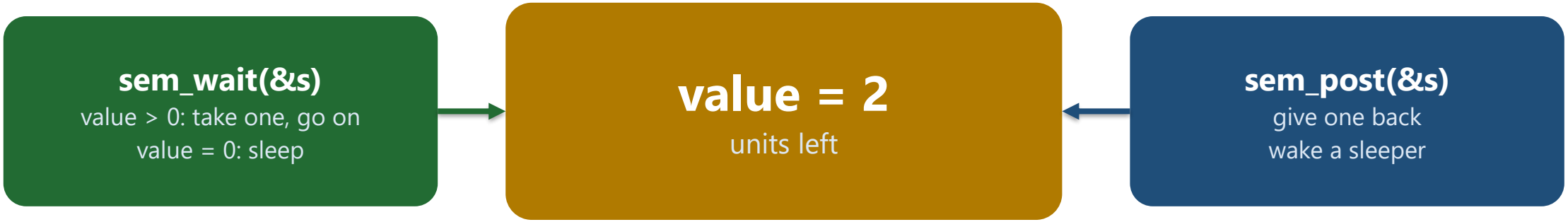
terminal (Linux, real)

```
$ time ./race
counter = 1084263 (expected 2000000)
real 0.00s
$ time ./mutex
counter = 2000000 (expected 2000000)
real 0.10s
$ time ./sem
counter = 2000000 (expected 2000000)
real 0.12s
```

- **race:** 0.00 s, and wrong
- **mutex:** 0.10 s, correct
- **sem:** 0.12 s, correct
- **why:** lock and unlock cost time, and the threads now take turns

QUESTION 2 · STEP 4

A semaphore: a number that never goes below zero



start 1 = a LOCK
`sem_init(&s, 0, 1)`

start 0 = a SIGNAL
`sem_init(&s, 0, 0)`

start N = a COUNTER
`sem_init(&s, 0, N)`

Same two calls. The start value decides what the semaphore is for.

EXERCISE 3

sem.c: a semaphore that starts at 1 is a lock

C (sem.c)

```c
sem_t s;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        sem_wait(&s);     // 1 -> 0
        counter++;
        sem_post(&s);     // 0 -> 1
    }
}

sem_init(&s, 0, 1);  // in main
```

terminal (Linux, real)

```
$ ./sem
counter = 2000000 (expected 2000000)
```

sem_init(&s, 0, 1): the 0 means "shared by threads", the 1 is the start value.

QUESTION

In sem.c we write sem_init(&s, 0, 2). What happens?

1

still 2000000

2

the race comes back

3

it hangs

4

it does not compile

THE ANSWER

In sem.c we write sem_init(&s, 0, 2). What happens?

1	2	3	4
still 2000000	**the race comes back**	it hangs	it does not compile

Value 2 lets BOTH threads in at once: two in the critical section, the race is back.

EXERCISE 4

order.c: B must wait until A has written

C (order.c)

```c
char message[32];
sem_t ready;              // starts at 0

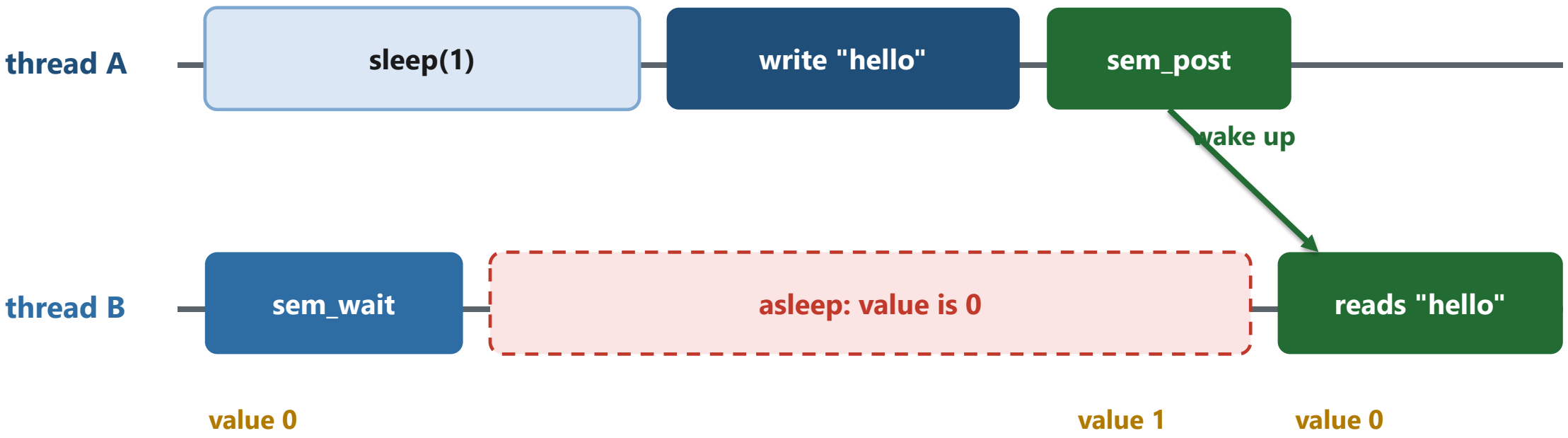
void *writer(void *arg) {
    sleep(1);
    strcpy(message, "hello");
    sem_post(&ready);     // 0 -> 1
}

void *reader(void *arg) {
    sem_wait(&ready);     // waits at 0
    printf("B: I read \"%s\"\n", message);
}
```

- **the problem:** not two at once, but the ORDER
- **sem_init(&ready, 0, 0):** nothing is ready at the start
- **sem_wait:** B sleeps here until A posts
- **sem_post:** A says: it is ready

QUESTION 2 · STEP 6

A semaphore at 0: a closed door until A opens it



Start at 0 means: nothing is ready yet. sem_post is the signal "it is ready".

EXERCISE 4 · OUTPUT

order: the real output

terminal (Linux, real)

```
$ ./order
B: waiting for the message...
A: message written
B: I read "hello"
```

- **line 1:** B starts first and goes to sleep
- **one second:** nothing happens: B uses no CPU
- **lines 2, 3:** A posts, B wakes and reads

QUESTION

order.c with sem_init(&ready, 0, 1). What does B print?

1

I read "hello"

2

I read "" (empty)

3

nothing, it hangs

4

an error

THE ANSWER

order.c with sem_init(&ready, 0, 1). What does B print?

1	2	3	4
I read "hello"	**I read "" (empty)**	nothing, it hangs	an error

At 1 the door is already open: B passes at once and reads before A has written.

EXERCISE 5

parking.c: 6 cars, 3 places

C (parking.c, key lines)

```c
#define PLACES 3
sem_t places;                 // free places

void *car(void *arg) {
    sem_wait(&places);        // take a place
    printf("car %ld enters\n", id);
    sleep(1);                  // parked
    printf("car %ld leaves\n", id);
    sem_post(&places);        // give it back
}

sem_init(&places, 0, PLACES);
```

- **start value 3:** three free places
- **cars 1, 2, 3:** take 3 to 0 and enter
- **car 4:** finds 0 and waits
- **the full file:** also counts "inside", under a mutex

EXERCISE 5 · OUTPUT

parking: never more than 3 inside

terminal (Linux, real)

```
$ ./parking
car 1 enters  (inside: 1)
car 2 enters  (inside: 2)
car 3 enters  (inside: 3)
car 1 leaves  (inside: 2)
car 2 leaves  (inside: 1)
car 4 enters  (inside: 2)
car 5 enters  (inside: 3)
car 3 leaves  (inside: 2)
car 6 enters  (inside: 3)
car 4 leaves  (inside: 2)
car 5 leaves  (inside: 1)
car 6 leaves  (inside: 0)
```

- **first:** cars 1, 2, 3 enter at once
- **then:** 4, 5, 6 enter only when someone leaves
- **the maximum:** inside: 3, guaranteed by sem_wait

QUESTION

parking.c with PLACES = 1. What is the semaphore now?

1

a signal

2

a lock (mutex)

3

useless

4

a deadlock

THE ANSWER

parking.c with PLACES = 1. What is the semaphore now?

1	2	3	4
a signal	**a lock (mutex)**	useless	a deadlock

A counter of 1 lets exactly one car in: a semaphore at 1 is a lock.

QUESTION 2 · STEP 10

Producer and consumer: three tools at once



empty = 1 (free slots)
the producer waits here when it is 0

full = 3 (filled slots)
the consumer waits here when it is 0

empty + full = 4, always. Two counters for waiting, one mutex for the buffer.

EXERCISE 6

prodcons.c: the two loops

C (prodcons.c, the loops)

```c
// producer
sem_wait(&empty);           // a free slot?
pthread_mutex_lock(&lock);
buf[in] = item; in = (in + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&full);            // one more to take

// consumer
sem_wait(&full);            // something to take?
pthread_mutex_lock(&lock);
item = buf[out]; out = (out + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&empty);           // one more free
```

- **empty:** starts at SIZE = 4
- **full:** starts at 0
- **mirror:** each side waits on one counter and posts the other
- **order:** semaphore first, mutex second

EXERCISE 6 · OUTPUT

prodcons: the buffer never overflows

terminal (Linux, real)

```
$ ./prodcons
put  1   (in buffer: 1)
put  2   (in buffer: 2)
put  3   (in buffer: 3)
put  4   (in buffer: 4)
            take 1   (in buffer: 3)
put  5   (in buffer: 4)
            take 2   (in buffer: 3)
put  6   (in buffer: 4)
            take 3   (in buffer: 3)
put  7   (in buffer: 4)
            take 4   (in buffer: 3)
put  8   (in buffer: 4)
            take 5   (in buffer: 3)
            take 6   (in buffer: 2)
            take 7   (in buffer: 1)
            take 8   (in buffer: 0)
```

- **put 1..4:** the fast producer fills the 4 slots
- **put 5:** only after take 1 frees a slot
- **never:** more than 4 in the buffer
- **the end:** the consumer empties it to 0

EXERCISE 6 · THE PRODUCER

Put the producer's steps in order

What is the right order?

A

```
lock(&lock)
```

B

```
sem_post(&full)
```

C

```
sem_wait(&empty)
```

D

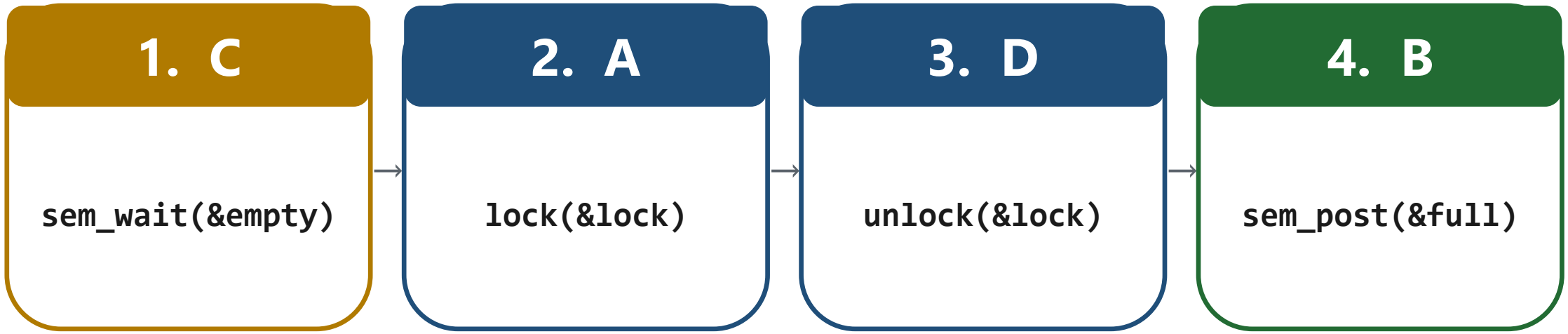
```
unlock(&lock)
```

Four calls around buf[in] = item. Which comes first?

EXERCISE 6 · THE PRODUCER

Put the producer's steps in order

The right order



Wait for space, lock, put, unlock, announce.

EXERCISE 6 · THE TRAP

Find the bug

prodcons.c

```
// producer, two lines swapped
pthread_mutex_lock(&lock);
sem_wait(&empty);
buf[in] = item; in = (in + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&full);
```

The buffer is full. What happens?

1. it works the same
2. the buffer overflows
3. everything hangs forever
4. the consumer skips items

EXERCISE 6 · THE TRAP

Find the bug

prodcons.c

```
// producer, two lines swapped
pthread_mutex_lock(&lock);
sem_wait(&empty);
buf[in] = item; in = (in + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&full);
```

The producer sleeps in sem_wait HOLDING the mutex; the consumer needs it to free a slot: deadlock.

1. it works the same
2. the buffer overflows
3. **everything hangs forever**
4. the consumer skips items

WHAT IT MEANS, CONCRETELY

The condition variable

The idea. A semaphore remembers a number. A condition variable remembers nothing: it is only a place to sleep until another thread says "something changed, look again".

Always with a mutex. pthread_cond_wait(&c, &m) releases the mutex, sleeps, and takes the mutex again before it returns.

Always with a while. When you wake up, the condition may already be false again (another thread was faster). So you check it in a loop, never with if.

TERMS

pthread_cond_wait: unlock, sleep, lock again

pthread_cond_signal: wake one waiter

pthread_cond_broadcast: wake all of them

Remember: **Condition variable = mutex + a condition you test in a while + a place to sleep.**

EXERCISE 7 · A

condvar.c: order.c with a condition variable

C (condvar.c, key lines)

```c
// reader (B)
pthread_mutex_lock(&lock);
while (!ready)
    pthread_cond_wait(&changed, &lock);
printf("B: I read \"%s\"\n", message);
pthread_mutex_unlock(&lock);

// writer (A)
pthread_mutex_lock(&lock);
strcpy(message, "hello"); ready = 1;
pthread_cond_signal(&changed);
pthread_mutex_unlock(&lock);
```

terminal (Linux, real)

```
$ ./condvar
B: not ready, I sleep
A: message written
B: I read "hello"
```

The condition is a normal variable (ready); the condition variable is only where B sleeps.

QUESTION 2 · STEP 14

The toolbox, on one page

tool	what it is	use it for
mutex	a lock with one key and an owner	protecting a critical section
semaphore	a counter that never goes below 0	signals, counting free resources
condition variable	a place to sleep until a condition changes	waiting for any condition, with a mutex
read-write lock	pthread_rwlock_t	many readers or one writer
atomic operation	atomic_fetch_add	one counter, no lock at all
spinlock	a lock you wait for by spinning	very short waits, inside kernels
barrier	pthread_barrier_t	everyone waits until all arrive

EXERCISE 7 · B

deadlock.c: two locks, opposite order

C (deadlock.c, lock = pthread_mutex_lock)

```c
void *one(void *arg) {
    lock(&A); sleep(1);
    lock(&B);   // waits for two
    ...
}

void *two(void *arg) {
    lock(&B); sleep(1);
    lock(&A);   // waits for one
    ...
}
```

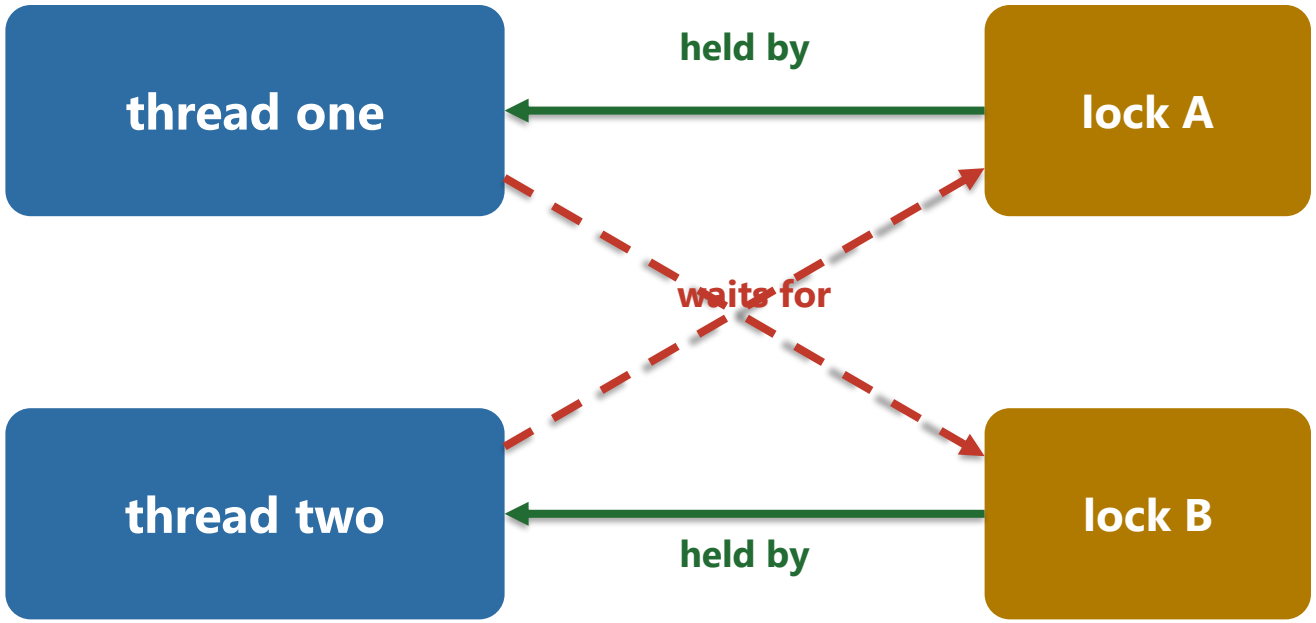
terminal (Linux, real)

```
$ timeout 5 ./deadlock; echo "exit code: $?"
exit code: 124
# after the fix: lock A, then B in both
one: I have A, I want B
one: I have both
two: I have B, I want A
two: I have both
done
exit code: 0
```

124 = killed by timeout after 5 s: the program was stuck, not slow.

QUESTION 2 · STEP 15

Deadlock: each one holds what the other needs



the fix

one: lock A, then B
two: lock A, then B

same order everywhere:
no circle, no deadlock

Take locks always in the same order, and the circle cannot close.

QUESTION

Which tools stop a race, and how?

The answer uses three words:

`mutex` `semaphore` `order`

THE ANSWER

Which tools stop a race, and how?

1. A mutex makes a critical section: one thread inside at a time.
2. A semaphore is a lock (start 1), a signal (start 0) or a counter (start N).
3. Wait on the semaphore before the mutex, and take locks always in the same order.

QUESTION 3

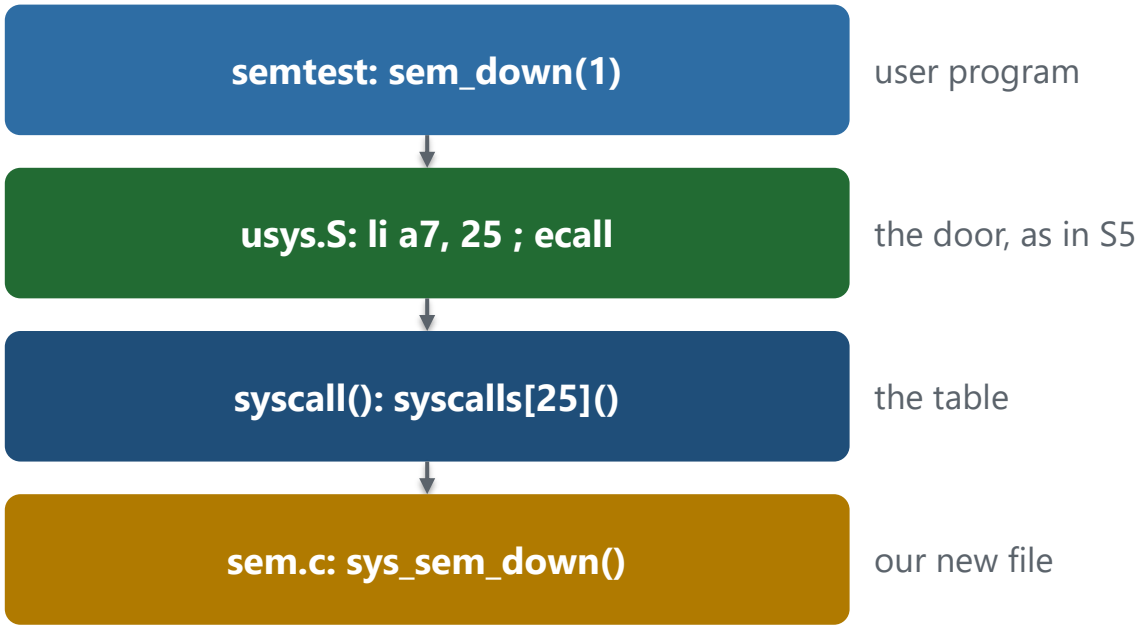
How does a kernel build a semaphore?

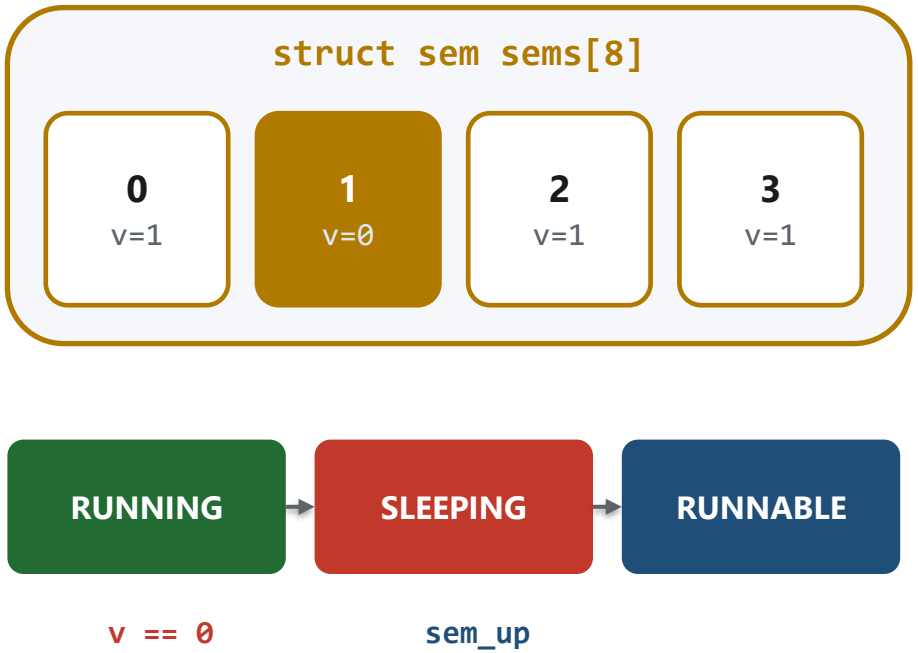
Exercise 8: three new system calls, sem_init (24), sem_down (25), sem_up (26).

The path · the spinlock · sem_down · the lost wakeup · sem_up · it runs

QUESTION 3 · STEP 1

A semaphore inside the kernel: the whole path





A kernel semaphore is a number, a spinlock, and the scheduler's sleep and wakeup.

WHAT IT MEANS, CONCRETELY

Spinning or sleeping?

A spinlock waits by looping on the CPU until the lock is free. Fine for a few instructions, wasteful for longer. In xv6 it also turns interrupts off on that CPU.

Sleeping means giving the CPU away: the process becomes SLEEPING and the scheduler runs someone else, until a wakeup.

A semaphore uses both. A spinlock for a few instructions, to read and change the number safely; sleep for the long wait.

TERMS

acquire / release: take / free a spinlock

sleep_prepare(chan): register what I wait for

sleep(): give the CPU away

wakeup(chan): make sleepers on chan runnable

Remember: **Spin for nanoseconds, sleep for anything longer.**

EXERCISE 8 · STEP 2

sem_down: the heart of the lab

C (kernel/sem.c, sys_sem_down)

```c
struct sem { struct spinlock lock; int value; };
static struct sem sems[NSEM];   // NSEM = 8

acquire(&s->lock);
while (s->value == 0) {
    sleep_prepare(s);    // I wait for s
    release(&s->lock);   // never sleep holding it
    sleep();             // give the CPU away
    acquire(&s->lock);
    if (killed(myproc())) { release(&s->lock); return -1; }
}
s->value--;
release(&s->lock);
return 0;
```

- **acquire:** only one CPU touches value at a time
- **while:** after waking up, check again
- **sleep_prepare(s):** the address of s is the channel
- **killed:** kill must work on a sleeper too

QUESTION 3 · STEP 4

Why sleep_prepare comes BEFORE release

WRONG order

CPU 1	value == 0, release(lock)
CPU 2	sem_up: value = 1, wakeup(s)
CPU 2	nobody registered: nobody wakes
CPU 1	sleep on s ... forever

xv6 order

CPU 1	value == 0, sleep_prepare(s)
CPU 1	release(lock)
CPU 2	sem_up: value = 1, wakeup(s): chan = 0
CPU 1	sleep(): chan is 0, return at once

Announce that you sleep while still holding the lock; only then let go.

EXERCISE 8 · STEP 4

sem_up, and the three places to register

C (kernel side)

```c
// kernel/sem.c
acquire(&s->lock);
s->value++;
wakeup(s);            // everyone sleeping on s
release(&s->lock);

// kernel/syscall.h
#define SYS_sem_init 24
#define SYS_sem_down 25
#define SYS_sem_up   26

// Makefile, OBJS
  $K/sem.o \
```

- **wakeup(s):** RUNNABLE again, they re-check the while
- **24, 25, 26:** 23 stays free for trace from Lab 10
- **sem.o:** a new kernel file must be in OBJS
- **the rest:** user.h, usys.pl, syscall.c: as in S5

EXERCISE 8 · STEP 5

semtest: the user program

C (user/semtest.c)

```c
sem_init(0, 0);                  // test 1: order
if (fork() == 0) {
    sem_down(0);
    printf("child: I ran after the parent\n");
    exit(0);
}
pause(10);
printf("parent: done, I let the child go\n");
sem_up(0);

sem_init(1, 1); sem_init(2, 0);  // test 2: ping-pong
// ping: down(1) print up(2)   pong: down(2) print up(1)
```

- **processes:** xv6 has no threads: fork, and the semaphores are shared by all
- **test 1:** order.c again, in xv6
- **test 2:** two signals passing the turn

EXERCISE 8 · CHECK 1

Before the kernel side: chaos

xv6 in QEMU, real (user side only)

```
$ semtest
3 semtest: unknown sys call 24
4 semtest: unknown sys call 25
child: I ran after the parent
parent: done, I let the child go
3 semtest: unknown sys call 26
3 semtest: unknown sys call 24
3 semtest: unknown sys call 24
# ... ping and pong mixed with errors
```

- **unknown sys call 24:** the stubs work, the kernel does not know 24 yet
- **child / parent:** the child prints BEFORE the parent
- **the lesson:** without synchronization the order is luck

EXERCISE 8 · CHECK 2

With the kernel side: order

xv6 in QEMU, real (CPUS=3)

```
$ semtest
parent: done, I let the child go
child: I ran after the parent
ping 1
  pong 1
ping 2
  pong 2
ping 3
  pong 3
semtest: OK
```

- **line 1, 2:** the child slept in sem_down until sem_up
- **ping / pong:** strict alternation, three times
- **semtest: OK:** the minimum to pass Lab 13

EXERCISE 8 · PART 3

mix: one letter per write(), two CPUs

xv6 in QEMU, real

```
$ mix
AAAAAAAbAbbAbAbbbbbbAbAbAbbAbAbbbbbbbbbbbbbbbbb
AbAbbbbbbbAbbAbAbAbAbAbbbbbbbbbbbbbbbbbAb
AAAAbbAA
bbbbbbbbbbbAbAAAAbbbAbAbAbAbAbAbAbAbAbAbAbAbA
# ...
$ mix lock
AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
# ...
```

- **mix:** letters of two processes mixed
- **mix lock:** sem 3 at 1 around each line: lines whole
- **all A first:** the parent takes the lock again at once: no queue, no fairness

QUESTION

Why "while (s->value == 0)" and not "if"?

1

while is faster

2

wakeup wakes everyone; another may take the unit first

3

if does not compile in the kernel

4

no reason, both work

THE ANSWER

Why "while (s->value == 0)" and not "if"?

1

while is faster

2

wakeup wakes everyone; another may take the unit first

3

if does not compile in the kernel

4

no reason, both work

Two sleepers, one sem_up: both wake, one takes the unit, the other must see 0 again and go back to sleep.

TRUE OR FALSE?

The kernel semaphore

1. A process waiting in sem_down keeps using the CPU.
2. The spinlock protects the number value.
3. It is fine to call sleep() while holding the spinlock.

THE ANSWERS

The kernel semaphore

1 **A process waiting in sem_down keeps using the CPU.**

FALSE. It is SLEEPING: the scheduler gives the CPU to others.

2 **The spinlock protects the number value.**

TRUE. Two CPUs could otherwise change it at the same time: the race of exercise 1, inside the kernel.

3 **It is fine to call sleep() while holding the spinlock.**

FALSE. sem_up needs the same spinlock: nobody could ever wake you. Deadlock.

QUESTION

How does a kernel build a semaphore?

The answer uses three words:

`spinlock` `sleep` `wakeup`

THE ANSWER

How does a kernel build a semaphore?

1. A number, protected by a spinlock so only one CPU changes it at a time.
2. down: while the number is 0, register, release, sleep; then take one.
3. up: add one and wakeup everyone sleeping on that semaphore.

IN THE LARGE

The same tools in real operating systems

	xv6	Linux	Windows NT
spin lock	struct spinlock	spinlock_t	KSPIN_LOCK
sleeping lock	sleeplock	struct mutex	ERESOURCE, fast mutex
semaphore	yours: sem.c	sem_t (on futex)	CreateSemaphore
user-space mutex	none	pthread_mutex (futex)	SRWLOCK, CRITICAL_SECTION
condition variable	sleep / wakeup	pthread_cond_t	CONDITION_VARIABLE

IN THE LARGE · LINUX

The futex: lock without the kernel, mostly

terminal (Linux, real)

```
$ strace -f -c -e trace=futex ./mutex
counter = 2000000 (expected 2000000)
# strace -c summary, trimmed
calls    errors   syscall
1204     446      futex
```

- **lock + unlock:** 2 000 000 pairs in mutex.c
- **system calls:** only 1 204 futex
- **the trick:** free lock: one atomic instruction in user space
- **only on contention:** ask the kernel to sleep: futex()

WHAT IT MEANS, CONCRETELY

Why synchronization bugs matter

They hide. A race shows once in a million runs, on a busy machine, at a customer. Tests rarely catch it.

They hurt. Lost money in a bank counter, a frozen server (deadlock), corrupted files: all the same bug as exercise 1.

They are attacked. TOCTOU (time of check to time of use): a program checks a file, an attacker swaps it before the use. Dirty COW (2016) was a race in the Linux kernel that gave root.

TERMS

TOCTOU: check, then use: the gap is a race

Dirty COW: CVE-2016-5195, a kernel race to root

ThreadSanitizer: gcc -fsanitize=thread finds races

Remember: **A race is not a slow bug, it is a wrong bug: and sometimes a door for an attacker.**

THIS WEEK IN THE LAB

Lab 11, Lab 12 and Lab 13

Lab 11 (Linux)

`race · mutex · sem · order · parking`

the race and the semaphore as a lock, a signal and a counter

Lab 12 (Linux)

`prodcons · condvar · readers · deadlock`

the tools together, and one deadlock built and removed

Lab 13 (xv6)

`sem_init · sem_down · sem_up`

the three calls (24, 25, 26), kernel/sem.c, then semtest and mix

Minimum to pass: Lab 11 Parts 1 to 3, Lab 12 Parts 1 and 2, semtest: OK in Lab 13.

WHAT STAYS

Three questions, three answers

Why is work lost?	Which tools?	How does a kernel do it?
counter++ is load, add, store; two threads interleave them.	Mutex for one-at-a-time; semaphore as lock (1), signal (0) or counter (N).	A number, a spinlock, and sleep / wakeup.

Next week: where a program's memory comes from: page tables.

THE WORDS OF THE DAY

Today's dictionary

term	in one sentence
thread	a flow of execution inside a process, sharing its memory
race condition	the result depends on who runs first
critical section	code that only one thread may run at a time
mutual exclusion	at most one inside the critical section
mutex	a lock: lock takes the key, unlock gives it back
semaphore	a number >= 0 with wait (take one) and post (give one)
condition variable	sleep until signalled, always with a mutex and a while
deadlock	a circle of threads, each waiting for the next
sleep / wakeup	how a kernel parks a process and brings it back

Thank you

Questions are welcome, now and any time.

Marius Dumitrașcu

UTM, FCIM / FAF

marius@cybercor.org · Telegram +40753210000